

Cyber Attacks Explained

Cryptographic Attacks

Cryptographic solutions are used to encrypt data transmission over wireless or wired protocols. Unfortunately, these techniques have proved to be vulnerable to attacks and data can be stolen. This article explores the various means to strengthen encryption techniques to protect network infrastructure—including methods using FOSS-based solutions.



As we all know, the heart of cryptographic network communication is public-key cryptography (PKI), which is used to encrypt the TCP/IP communication between two network end-points. PKI uses various encryption algorithms to ensure data security. The whole idea behind encryption is to make it a very difficult, time-consuming task to try out all possible keys. For example, if a message is encrypted using an 8-bit key, it means that 256 different combinations of the key need to be tried to decrypt the data. Any computer can perform this task in less than a second. However, if the key length is extended to 32, it would need 65,536 combinations to be tried, needing a few seconds. Extending this, the number of combinations for a 256-bit key would need literally many years for even a powerful computer to crack.

While the key length is an important factor, the mathematical algorithm used for processing encryption and decryption is also equally important. The algorithm is supposed to quickly perform the action, while maintaining necessary data and key security. There are many algorithms, such as SHA1, 3DES, etc. There are two types of keys—symmetric and asymmetric. The symmetric types use only one key for encryption and decryption, while for asymmetric keys, there are two different keys, which are complementary to each other. Please refer to Figure 1, which shows the basic cryptography functionality, which is designed with the objectives of data confidentiality, integrity and authentication in mind.

It is important to understand what cryptography means to the Internet. The Internet is blessed with SSL (Secure Socket Layer) and TLS (Transport Layer Security) protocols, which perform the job of encrypting and decrypting the data sent, so as to enable users to securely exchange personal

information, credit card numbers, etc. SSL and TLS are based on asymmetric keys exchange. There are two demands here—that the data must be encrypted for security reasons, and that the website must be known to be legitimate. The latter is important because attackers may host websites to steal personal information. To ensure legitimacy, the Web server has an SSL certificate, which enables traffic via the HTTPS protocol, using TCP port 443 for communication. This SSL certificate is provided or signed by a trusted certification authority such as Verisign or Thawte, which ensures that the SSL certificate holder is a genuine party, will adhere to security standards, and is eligible to obtain a certificate and install it on their servers. The SSL certificate is tied to the domain name, such as *abcd.com*.

To understand all the security concerns, we need to first study how certificates work. Digital certificates using asymmetric PKI have two keys, a public key and a private key. The private key is installed on the Web server where the website URL is supposed to be secured using SSL. The public key is shipped with all browsers that support SSL. To support multiple certificate authority vendors, browsers are equipped with their public keys, as well as various ciphers (the encryption and decryption algorithms). Each public key has an expiration date and needs to be updated once it nears expiration. When we install a digital certificate on a Web server, we are essentially installing the private key, which is specifically created by the trusted certification-providing authority.

Now let's see how this mechanism works at a higher level, for a browser. As shown in Figure 2, when a person tries to access an SSL-secured website, the browser first challenges the server by sending its own cipher strength. In response, the server does the same, and also sends a copy